upGrad

Capstone Workflow: Casting Defect Detection

Introduction

This capstone focuses on building a complete, reproducible Deep-Learning MLOps system that inspects manufactured castings and classifies each as OK or Defective from an image. The emphasis is on the full operational lifecycle of a computer-vision model in production — not just training a CNN, but operating it responsibly: validate → version → train → track → register → deploy → monitor → retrain → govern.

Learners design and implement an end-to-end workflow that begins with framing the visual-inspection objective, validating image data quality and versioning the dataset, then trains a transfer-learning CNN (ResNet18) with MLflow tracking and a model registry, serves it through a FastAPI image endpoint, containerises it with Docker, validates it with GitHub Actions CI, and finally closes the loop with statistical and embedding drift monitoring and a drift-triggered retraining workflow that compares model versions and **rolls back** safely.

The progressively enhanced system spans:

- **Baseline transfer-learning model** — ResNet18 (frozen backbone + new 2-class head) — Solution V1
- **Drift-augmented retrained model** — promoted only if it beats production, else rolled back — Solution V2

Business Problem

Manual visual inspection of manufactured parts is slow, costly and inconsistent. A computer-vision system can automatically flag defective castings on the line, increasing throughput and consistency — but a model alone is not enough to deliver value in production.

Cameras, lighting conditions and product lines change over time, so the system must be deployed, monitored and maintained: it must detect when incoming images drift away from the training distribution and retrain safely, with the ability to roll back a bad model. The core difficulty is twofold.

- First, the cost of errors is asymmetric — a missed defect (false negative) ships a bad part, so the system must be tuned for recall on defects, not bare accuracy.
- Second, image distributions shift in ways raw-pixel checks miss, so monitoring must look at both interpretable image statistics and the model's own feature space (embeddings).

This capstone therefore frames the problem as an MLOps problem: produce a model and the operational machinery that lets a factory trust, serve, monitor and maintain it.

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.

Data Description

Casting Product Image Data — top-view images of submersible-pump impeller castings, captured under stable factory lighting.

1. Structure and Classes

- **Approximately 7,348 grayscale JPEGs (300×300)**, pre-split into train/ and test/ with two class folders.
- **Classes:** ok_front (acceptable) vs def_front (defective — blowholes, pinholes, burr, shrinkage).
- **Defect = positive class** — a missed defect is the costly error, so **recall on defects** is the headline metric (with F1 and ROC-AUC).

2. Data Quality and Constraints (assess before modelling)

- **Integrity:** Programmatic checks for missing/corrupt files and consistent image dimensions before any split.
- **Duplicates:** Detect duplicate images via **hashing** (representative ~64 flagged) — duplicates across train/test would leak.
- **Class balance:** Quantify the class distribution and produce a pass/fail data-quality summary.
- **Versioning:** Snapshot each dataset split (file lists + metadata.json: version id, date, split sizes, seed) for auditability and reproducibility.
- **Backbone compatibility:** Images are grayscale and must therefore be replicated across three channels, resized to 224 × 224, and normalised using ImageNet statistics to match the pretrained ResNet18 input requirements.
- **Compute:** CPU-only; transfer learning (frozen backbone) and a stratified subsample keep it laptop-runnable.

Overall Methodology and Learning Outcomes

The project follows a structured DL/MLOps lifecycle emphasising reproducibility, leakage-safety, honest evaluation and evidence-based operation, organised into **four stages delivered as four weekly milestones**.

Stage 1 — Business & Data Understanding, Quality, Versioning, Methodology

Frame the visual-inspection objective and the right metric (defect recall, not accuracy); validate image data quality (corrupt/duplicate/dimension/distribution); create reproducible, versioned, stratified splits; perform image EDA; and design the end-to-end MLOps workflow.

Stage 2 — Preprocessing, Transfer Learning & Experiment Tracking

Build reproducible augmented inputs (augmentation on train only); configure an ImageNet-pretrained ResNet18 (frozen backbone + new 2-class head); train with a class-weighted loss, validation evaluation and early stopping; and track parameters, per-epoch metrics and the model in MLflow.

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.

Stage 3 — Evaluation, Registry, Deployment & CI/CD

Evaluate with imbalance-aware metrics and failure analysis; register the best model and promote it to @production; serve predictions via a FastAPI image endpoint; containerise with Docker; and validate with GitHub Actions CI.

Stage 4 — Monitoring, Drift, Retraining & Governance

Log predictions and monitor confidence; detect statistical drift (Evidently + PSI on image features) and embedding drift (feature-space PSI); define retraining triggers; retrain a drift-augmented candidate; and compare versions and roll back safely — capturing all operational evidence inside the submitted notebook and report.

Learning Outcomes

By completing this capstone you will be able to:

1. Define DL/MLOps objectives, success criteria and deployment constraints for a visual-inspection problem.
2. Validate image data quality and build reproducible, versioned, augmented train/val/test splits.
3. Configure transfer learning with a pretrained CNN (ResNet18) in PyTorch (frozen backbone + trained head).
4. Track experiments with MLflow; register and promote models.
5. Evaluate with imbalance-aware metrics (recall on defects, F1, ROC-AUC, confusion) and analyse failures.
6. Serve via FastAPI, containerise with Docker, automate with GitHub Actions.
7. Detect statistical drift (Evidently + PSI) and embedding drift (feature-space PSI), and monitor confidence.
8. Define retraining triggers; retrain, compare versions, and roll back safely — and capture operational evidence inside submittable artefacts (notebooks/report).

Evaluation Metrics

The headline metric is recall on the defect class (with F1 and ROC-AUC), chosen because of this problem:

- **Recall on defects:** A missed defect ships a bad part; this is the costly error to control.
- **F1 / ROC-AUC:** Balance precision against recall and give a threshold-independent view.
- **Confusion matrix + failure analysis:** Expose which defects are missed.
- **Accuracy is explicitly NOT the selection metric:** It hides missed defects. Justify your metric from the problem and cost structure, not by habit.

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.

Project Stages

Stage 1 — Business & Data Understanding, Quality, Versioning, Methodology [20 Marks]

Task 1.1 — Business & Data Understanding [5 Marks]

- **1.1.1 — Business objective & justification [2]** — clear objective (automated defect detection) justified by cost of a missed defect / throughput.
- **1.1.2 — Success metric choice (recall/F1 on defects) [2]** — problem-appropriate metrics rather than accuracy alone, justified by the asymmetric cost.
- **1.1.3 — Dataset structure & class understanding [1]** — ~7,348 images, 2 classes, defect = positive.

Task 1.2 — Data Quality Validation [5 Marks]

- 1.2.1 — Corrupt-file & image-dimension checks [2]
- 1.2.2 — Duplicate detection (hashing) — leakage guard [2]
- 1.2.3 — Class distribution + pass/fail report [1]

Task 1.3 — Dataset Versioning & Metadata [5 Marks]

- 1.3.1 — Reproducible, stratified train/val/test split [3]
- 1.3.2 — Split snapshot + metadata.json (versioning) [2]

Task 1.4 — EDA & Methodology Design [5 Marks]

- 1.4.1 — Image-characteristic EDA [3] — sample grid, class distribution, per-class feature statistics.
- 1.4.2 — End-to-end MLOps workflow design [2]

Stage 2 — Preprocessing, Transfer Learning & Experiment Tracking [25 Marks]

Task 2.1 — Preprocessing & Augmentation [6 Marks]

- 2.1.1 — Grayscale replication to three channels, resizing to 224 × 224, and ImageNet normalisation [3]
- 2.1.2 — Train-only augmentation (val/test none) [3]

Task 2.2 — Transfer-Learning Model [7 Marks]

- 2.2.1 — ImageNet-pretrained ResNet18, backbone frozen [4]
- 2.2.2 — New 2-class head replaces fc (head trained) [3]

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.

Task 2.3 — Training Workflow [6 Marks]

- 2.3.1 — Class-weighted loss, optimiser configuration, trainable parameters, and random seed [3]
- 2.3.2 — Epoch loop + validation eval + early stopping [3]

Task 2.4 — MLflow Experiment Tracking [6 Marks]

- 2.4.1 — Params + per-epoch metrics logged [3]
- 2.4.2 — Trained model logged to the run [3]

Stage 3 — Evaluation, Registry, Deployment & CI/CD [25 Marks]

Task 3.1 — Evaluation & Failure Analysis [7 Marks]

- 3.1.1 — Imbalance-aware metrics + model selection [4]
- 3.1.2 — Confusion matrix and ROC visualisations + failure-case analysis [3]

Task 3.2 — Model Registry & Promotion [5 Marks]

- 3.2.1 — Best model registered in MLflow Registry [2]
- 3.2.2 — Promotion to production alias + version history [3]

Task 3.3 — FastAPI Inference Service [6 Marks]

- 3.3.1 — Image /predict + /health + logging + validation [4]
- 3.3.2 — Live call demonstrated in-notebook [2]

Task 3.4 — Containerisation & CI/CD [7 Marks]

- 3.4.1 — Valid Dockerfile [3]
- 3.4.2 — GitHub Actions CI + passing-test evidence [4]

Stage 4 — Monitoring, Drift, Retraining & Governance [30 Marks]

Task 4.1 — Prediction Logging & Confidence Monitoring [5 Marks]

- 4.1.1 — Prediction logging with timestamp [2]
- 4.1.2 — Confidence monitoring for reference and current batches, including threshold definition [3]

Task 4.2 — Statistical Drift Detection [8 Marks]

- 4.2.1 — Per-image features + Evidently DataDriftPreset [4]
- 4.2.2 — PSI per feature + interpretation [4]

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.

Task 4.3 — Embedding Drift Detection [7 Marks]

- 4.3.1 — 512-dim embedding extraction (reference + current) [3]
- 4.3.2 — Feature-space drift quantified (PSI) + explanation [4]

Task 4.4 — Retraining Workflow & Triggers [6 Marks]

- 4.4.1 — Measurable retraining triggers [2]
- 4.4.2 — Drift-triggered retraining + new version registered [4]

Task 4.5 — Version Comparison & Rollback Governance [4 Marks]

- 4.5.1 — Candidate vs production comparison on drifted batch [2]
- 4.5.2 — Promote-on-improvement-else-rollback gate + decision recorded [2]

Deliverables

#	Deliverable	Format	Covers
1	Casting_Defect_MLOps_Reports	PDF	Stages 1, 3, 4 — business, methodology, results, monitoring, governance + **Stage-4 operational evidence** (MLflow/Docker/CI/drift screenshots + summaries)
2	Data_Preparation.ipynb	Executed notebook	Stage 1 (understanding/quality/versioning/EDA) + Stage 2.1 (preprocessing/augmentation)
3	Model_Development_and_Tracking.ipynb	Executed notebook	Stage 2.2–2.4 (transfer learning + MLflow) + Stage 3 (evaluation, registry, FastAPI/Docker/CI evidence)
4	Operations_Monitoring_and_Evidence.ipynb	Executed notebook	Stage 4 — logging/confidence, statistical drift, embedding drift, retraining, version comparison & rollback
5	app.py	Python script	Stage 3 FastAPI inference service source

Note: Model artefacts (.pt/.pth, MLflow store, Docker images, drift HTML, embedding .npz) are NOT required. Evidence is captured through notebook implementation, in-notebook outputs, listings, screenshots, training/prediction logs, and reproducibility documentation in the report.

Tools, Libraries, and Technology Stack

This capstone implements an end-to-end Deep Learning MLOps workflow for automated casting defect detection using transfer learning, experiment tracking, deployment, monitoring, and retraining pipelines.

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.

Programming Environment

- Python 3.11+
- Jupyter Notebook or JupyterLab
- VS Code (optional)

Data Processing and Analysis

- pandas
- numpy
- scikit-learn
- matplotlib
- seaborn

Image Processing and Computer Vision

- Pillow (PIL)
- OpenCV (optional)
- torchvision.transforms

Deep Learning Framework

- PyTorch 2.x
- Torchvision
- Pretrained ResNet18 (ImageNet weights)

Experiment Tracking and Model Registry

- MLflow Tracking
- MLflow Model Registry

API Development and Deployment

- FastAPI
- Uvicorn

Containerisation and CI/CD

- Docker
- GitHub Actions

Monitoring and Drift Detection

- Evidently
- Population Stability Index (PSI)

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.

Supporting Libraries

- pathlib
- logging
- json
- hashlib
- joblib (optional)

The capstone is intentionally designed to be executable on CPU-only environments through transfer learning with a frozen ResNet18 backbone and stratified subsampling.

Implementation Guidance

Learners are advised to follow the implementation sequence below.

1. Complete all image data quality checks before model development:
   - Missing or corrupt image detection
   - Image dimension validation
   - Duplicate image detection
   - Class distribution analysis

2. Create reproducible, stratified train, validation, and test partitions before implementing preprocessing or augmentation steps.

3. Maintain reproducible experiments throughout the project:
   - random_state = 42
   - Fixed dataset partitions
   - Documented hyperparameters
   - Version-controlled artefacts

4. Apply preprocessing consistently across all experiments:
   - Grayscale replication to three channels
   - Resize images to 224 × 224
   - Apply ImageNet normalisation
   - Apply augmentation only on the training split

5. Establish the baseline transfer-learning system before implementing monitoring and retraining workflows.

6. Validate each component independently before integration:
   - Data preparation pipeline
   - ResNet18 training workflow

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.

- MLflow experiment tracking
- FastAPI inference service
- Docker container
- GitHub Actions workflow
- Monitoring and drift detection pipeline

7. Log and preserve operational evidence throughout implementation:
   - MLflow runs and model versions
   - Training metrics and visualisations
   - Prediction examples
   - API outputs
   - Docker execution evidence
   - CI execution evidence
   - Drift reports and monitoring outputs
   - Retraining and rollback decisions

8. Evaluate models using defect-focused metrics:
   - Recall on defects
   - F1 score
   - ROC-AUC
   - Confusion matrix and failure analysis

9. Compare candidate and production models using identical evaluation procedures before promotion or rollback decisions.

The emphasis of this capstone is on building a reproducible and maintainable computer-vision MLOps system rather than solely achieving the highest classification accuracy.

Common Mistakes and Troubleshooting

Data Understanding and Preparation

- Assuming the provided train and test folders are automatically leakage-safe.
- Failing to detect duplicate images across dataset splits.
- Ignoring corrupt image files and inconsistent image dimensions.
- Performing preprocessing before creating reproducible dataset partitions.

Image Preprocessing

- Forgetting to replicate grayscale images to three channels.
- Applying ImageNet normalisation incorrectly.
- Applying augmentation to validation or test datasets.

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.

- Using inconsistent preprocessing pipelines across experiments.

Transfer Learning and Training

- Training all ResNet18 layers instead of freezing the backbone.
- Replacing the classifier head incorrectly.
- Ignoring class imbalance during model training.
- Evaluating directly on the test set during development.
- Selecting models using accuracy alone instead of recall on defects.

Experiment Tracking

- Logging metrics without corresponding parameters.
- Failing to register the best model version.
- Overwriting previous experiments.
- Not recording model version history and production aliases.

Deployment and Operations

- Loading preprocessing separately from the trained inference pipeline.
- Creating FastAPI endpoints without input validation.
- Building Docker images that do not reproduce the local environment.
- Creating CI workflows that do not execute validation checks successfully.

Monitoring and Drift Detection

- Monitoring only statistical drift and ignoring embedding drift.
- Computing PSI values without interpretation.
- Defining retraining based on a single signal.
- Retraining models without recording comparison results and decisions.
- Promoting new models without evaluating against the current production version.

General Recommendation

Validate each stage independently before progressing to the next stage. Preserve intermediate artefacts, maintain experiment records, and capture operational evidence throughout implementation to ensure reproducibility and reliable operational decision-making.

Appendix 1 — Monitoring & Drift (operational design, with embedding-drift scaffolding)

Monitoring is assessed on operational clarity, not just on running a tool. Two complementary signals:

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.

a) **Statistical drift:** For each image, compute interpretable features (brightness, contrast, edge density, sharpness, dark-ratio). Compare the reference (training) distribution against a current batch using Evidently DataDriftPreset + PSI per feature. A feature with PSI > 0.2 has drifted.

b) **Embedding drift**
   a. **Feature extraction:** Take the model's penultimate layer (the 512-dim vector before the final classifier) as image enbedding (EmbeddingExtractor in src/model.py).
   b. **Embedding generation:** Run the reference set and the current batch through the backbone and collect their embedding vectors.
   c. **Feature-space comparison:**Reduce each embedding to a scalar distance to the reference centroid (mean embedding of the reference set), giving one distribution per batch.
   d. **Drift calculation:** Compute PSI between the reference and current distance distributions; PSI above the threshold (~0.10 for embeddings) signals feature-space drift even if raw pixels look similar.

- **Reference dataset:** reference_embeddings.npz / reference_features.csv saved at training time are the fixed baselines.
- **Cadence & action:** Score each incoming batch on a predefined schedule. If drift triggers fire, retrain the candidate model, compare it with production, and either promote or roll back the model accordingly.
- **Representative thresholds:** PSI > 0.20 (statistical drift) and PSI > 0.10 (embedding drift), and the confidence-drop alert are sensible instructional defaults; tune them to the line's tolerance in production.

Appendix 2 — Assumptions & Constraints

- Binary visual-inspection task; defect = positive class.
- CPU-based transfer learning using a frozen backbone keeps the capstone executable on standard laptops; a stratified training subsample is used for speed and scales to the full set.
- All preprocessing/augmentation is defined once and applied per split (augmentation on train only).
- Drift is demonstrated by simulated camera/lighting degradation on held-out images; the monitoring mechanism is production-ready.
- Reproducible (random_state/seed = 42); outputs saved to files and evidenced in the notebooks.

Appendix 3 — Glossary

- **Transfer learning:** Reuse a pretrained CNN backbone; train only a new classification head.
- **Embedding:** The model's penultimate-layer feature vector (512-dim for ResNet18) representing an image.

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.

- **Embedding drift:** Distribution shift measured in the model's feature space (PSI on distance-to-centroid).
- **PSI:** Population Stability Index; a binned distribution-shift measure. Representative thresholds are PSI > 0.20 for statistical drift and PSI > 0.10 for embedding drift.
- **Rollback:** Reverting the @production alias to a previous model version after a regression.
- **Registry alias:** A named pointer (@production) to the live model version in MLflow.

Disclaimer: *All content and material on the upGrad website is copyrighted material, belonging to either upGrad or its bonafide contributors, and is purely for the dissemination of education. You are permitted to access, print, and download extracts from this site purely for your own education only and on the following basis:*

- *You can download this document from the website for self-use only.*
- *Any copies of this document, in part or full, saved to disc or to any other storage medium, may be used for subsequent, self-viewing purposes or to print an individual extract or copy for noncommercial personal use only.*
- *Any further dissemination, distribution, reproduction, or copying of the content of the document herein, or the uploading thereof on other websites, or use of the content for any other commercial/unauthorized purposes in any way that could infringe the intellectual property rights of upGrad or its contributors, is strictly prohibited.*
- *No graphics, images, or photographs from any accompanying text in this document will be used separately for unauthorized purposes.*
- *No material in this document will be modified, adapted, or altered in any way.*
- *No part of this document or upGrad content may be reproduced or stored on any other website or included in any public or private electronic retrieval system or service without upGrad's prior written permission.*
- *Any rights not expressly granted in these terms are reserved.*

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.